

Homework 6

Uzair Hamed Mohammed

CSC 510, Spring 2026

Due 5/18

1. Calculate a depth-first search of the following two directed graphs using the “alphabetical order” convention for vertices and outgoing edges. For both graphs, draw the desulting DFS tree. For the first graph, indicate the type (back , forward , or cross) of the remaining edges. For the second graph, which is acyclic, write down the pre- and post-order traversals of the DFS. Use that information to find a topological sort of the graph. (Note: because of the alphabetical ordering, there is only one acceptable DFS tree for each graph, and only one acceptable topological sort for the second graph)

Sol:

Graph 1

DFS Tree:

- A
 - B
 - E
 - I
- C
 - F
 - J
 - K
 - H
 - D
 - G
 - L

Edge Types:

- Back: D -> C, G -> F, G -> K
- Forward: C -> G, C -> H, F -> K
- Cross: C -> B, E -> B, F -> B, F -> E, J -> E, J -> I, L -> H

Graph 2

DFS Tree:

- A
 - B
 - E
 - F
 - I
 - J
 - G
 - K
- C

- D
- H
- L

Traversals & Sort:

- Pre-order: A, B, E, F, I, J, G, K, C, D, H, L
- Post-order: B, F, K, G, J, I, E, A, D, L, H, C
- Topological sort: C, H, L, D, A, E, I, J, G, K, F, B

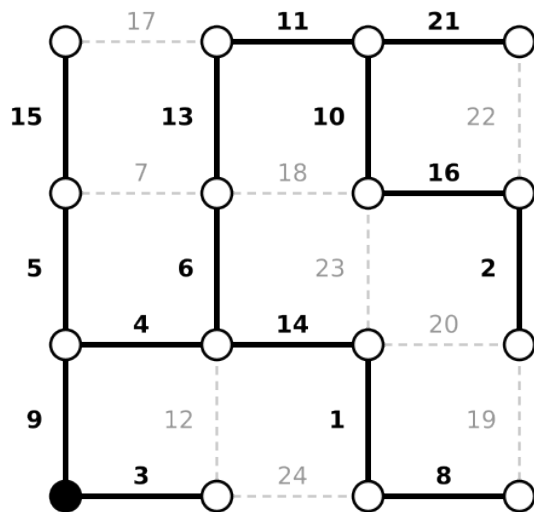
2. Run both Kruskal and Prim's algorithms on the following weighted graph, and draw the resulting minimum spanning tree. For Prim's algorithm, start at the bottom-left vertex marked in black. To show your work, list the weights of the edges in the order they are added to the MST by each algorithm. What's the weight of the minimum spanning tree?

Sol:

Kruskal's: 1, 2, 3, 4, 5, 6, 8, 9, 10, 11, 13, 14, 15, 16, 21

Prim's: 3, 9, 4, 5, 6, 13, 11, 10, 14, 1, 8, 15, 16, 2, 21

Total weight: $(1 + 2 + 3 + 4 + 5 + 6 + 8 + 9 + 10 + 11 + 13 + 14 + 15 + 16 + 21) = 138$



3. Consider all weighted, undirected, connected graphs where the edge weights are distinct (in particular, there is a unique MST) and every vertex has degree at least 2.
- a. Does the minimum spanning tree of every such graph always contain the lightest edge? Why or why not?

Sol:

The minimum spanning tree of every such graph always contains the lightest edge. This is because the lightest edge is the unique minimum-weight edge across the cut that separates its two endpoints. By the cut property, it must belong to every MST, regardless of the condition that every vertex has degree at least 2.

- b. Is there such a graph whose minimum spanning tree contains the heaviest edge? Why or why not?

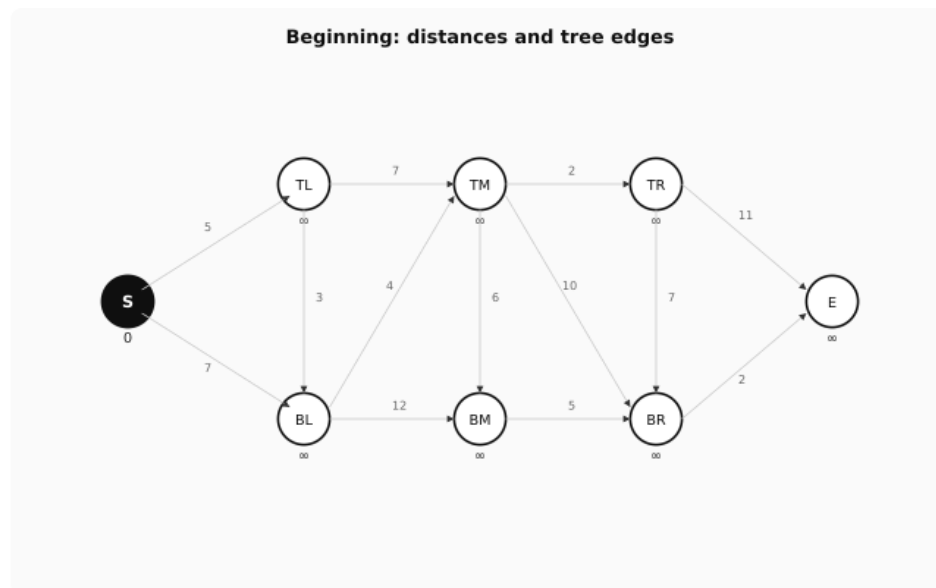
Sol:

Yes, there exists such a graph. For example, take two disjoint cycles (each vertex degree 2) and connect them by a single bridge edge. Assign the largest weight to this bridge. The graph is connected, all vertices have degree at least 2 (the bridge endpoints have degree 3), and all edge weights are distinct. Since the bridge is a cut edge, it must be included in every spanning tree, hence also in the MST. Therefore the MST contains the heaviest edge.

4. Run Dijkstra's algorithm on the following graph, starting from the left-most vertex marked in black. To show your work, draw the vertex labels and tree edges:

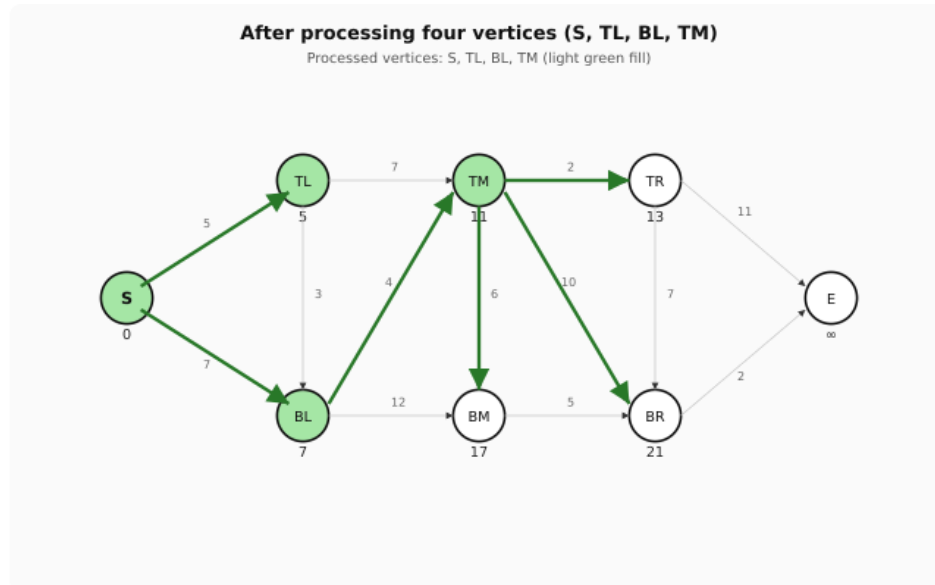
- a. at the beginning of the algorithm

Sol:



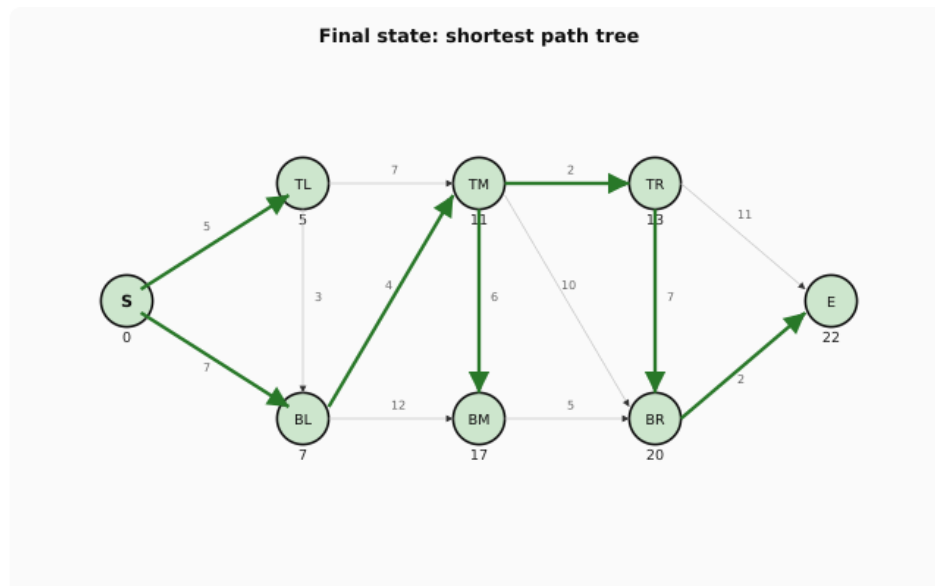
- b. after processing (i.e. relaxing the outgoing edges of) four vertices, and

Sol:



c. at the end of the algorithm.

Sol:



For (b), also indicate which vertices have been processed.